```
  ...

  private SecurityMockServerConfigurers.OidcLoginMutator
    configureMockOidcLogin(User expectedUser)
  {
    return mockOidcLogin().idToken(builder -> {
      builder.claim(StandardClaimNames.PREFERRED_USERNAME,
        expectedUser.username());
      builder.claim(StandardClaimNames.GIVEN_NAME,
        expectedUser.firstName());
      builder.claim(StandardClaimNames.FAMILY_NAME,
        expectedUser.lastName());
      builder.claim("roles", expectedUser.roles());     ❶
    });
  }
}
```

❶ Adds a “roles” claim to the mock ID Token

You can verify that the changes are correct by running the following command:

```
$ ./gradlew test --tests UserControllerTests
```

NOTE The `roles` claim configured in Keycloak will include our custom roles (`employee` and `customer`) together with a few extra roles managed and assigned by Keycloak itself.

So far, we’ve configured Keycloak to include the user roles in the tokens and updated Edge Service to relay the Access Token to the applications downstream. We’re now ready to start securing Catalog Service and Order Service with Spring Security and OAuth2.

12.2 Protecting APIs with Spring Security and OAuth2 (imperative)

When a user accesses the Polar Bookshop application, Edge Service initiates the OpenID Connect authentication flow through Keycloak and ultimately